

Contents

Tutorial 1	1	Tutorial 2	2
1 String Rotation	1	1 Maximum Disjoint s - t Paths	2

1. String Rotation A *rotation* of a string α is the string $\alpha[k :]\alpha[: k]$ (the last $|\alpha| - k$ characters of α preceding its last k) for some index k .

- Propose a linear-time algorithm which, given two strings α and β , decides whether α is a rotation of β for some k .
 - It should be relatively clear that a **True** response implies that α is a rotation of β . Prove the following:
Proposition. *If my process returns **False**, then α is not a rotation of β .*
 - Why does your process fail when looking for a rotation of β contained in α ? Give an example.
- ★★ How long might it take to find a rotation of β contained in α ?

- The time complexity of *my* solution is $\mathcal{O}(\min(|\alpha|, |\beta|))$.
 - Contraposition ($A \rightarrow B \equiv \neg B \rightarrow \neg A$)

- Verify that $|\alpha| = |\beta|$ before using the Knuth-Morris-Pratt algorithm to check for β as a substring of the concatenation $\alpha\alpha$. Verifying the sizes can be done in $\mathcal{O}(\min(|\alpha|, |\beta|))$ by iterating through both strings until one ends, then KMP will run in the same time complexity. The value k will be the index corresponding to the first entry of the substring.
 - Proof.* The contrapositive “If α is a rotation of β , then my process returns **True**” is logically equivalent to the proposition statement.

Notice that, if α is indeed a rotation of β , then $\alpha\alpha = \beta[k :]\beta[: k]\beta[k :]\beta[: k] = \beta[k :]\beta\beta[: k]$ must contain β as a substring regardless of the value of k . KMP would find this substring and return **True**. □
 - If $\alpha = \mathbf{f}\beta[k :]\beta[: k]$, then $\alpha\alpha = \mathbf{f}\beta[k :]\beta[: k]\mathbf{f}\beta[k :]\beta[: k]$ no longer contains β despite α containing a rotated β .

★★ A **False** instance requires searching for each of the $|\beta|$ rotations of β in α and necessarily takes quadratic time.

1. Maximum Disjoint s - t Paths Given a directed network with source s and sink t , we want to find a maximum collection of edge-disjoint, $s - t$ paths. Consider the following procedure:

- Run BFS rooted at s until t is reached for the first time.
- Identify the shortest $s - t$ path p by following parents back to s .
- Remove p from the graph and add it to P .
- Repeat until s and t lie in separate components.

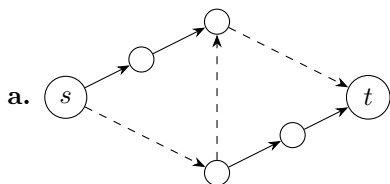
The set P is clearly a collection of edge-disjoint, $s - t$ paths since no edge can be added to P more than once.

- a. Provide an example network on which this procedure fails to produce a *maximum* solution.
- b. Propose a revised procedure which produces the correct set on any network and prove the following:

Proposition. *The collection produced by the revised procedure is maximum.*

- c. What if the paths have to be internally vertex-disjoint (except for s and t)?
- ★★ What needs to change for use on an undirected graph?

- a. Eventually, removing the shortest path disconnects the graph. Can this happen too early?
- b. Reduce P to a network flow. Proof by contradiction?



Breadth-first search always finds the shortest path from s to t . In this case, s and t are disconnected after only one path is identified despite two edge-disjoint paths being possible.

- b. Assign unit capacity to each edge, find a maximum $s - t$ flow and remove unsaturated edges before performing the original procedure. Notice that the central, vertical edge above will be removed by this pre-processing step and the correct, two-path solution will be found.

Proof. Clearly, any collection P can be converted into a unit-flow by saturating those edges which appear anywhere in P ; this flow will have value equal to the number of paths in P . It remains only to show that the collection P^* produced by the revised procedure translates into a maximum unit-flow.

Suppose, for the sake of contradiction, that P^* translates into a sub-optimal unit-flow. Since the flow F^* produced during the pre-processing step is necessarily maximum, this implies that there exists an edge e which is saturated by F^* but is not used by any path in P^* . Because both flow and elements of P^* must originate at s and end in t , we have a contradiction. Either e must be part of an $s - t$ path which was never added to P^* or the flow through e is imbalanced. $\square$

- c. Add unit capacity to each vertex. Ford-Fulkerson can still be used if we split each vertex into two vertices joined by a unit capacity edge.
- ★★ Treat undirected edges as two entangled edges in opposing directions; both have a capacity of one, but saturating one adds capacity to the other. Keep only those saturated directed edges for the second stage.